Practice Question

Question

A software company manages different types of workers: **full-time developers** (monthly salary plus performance bonus), **freelance developers** (paid per project), and **technical leads** (full-time developers who also work as freelance consultants on external projects).

Design and draw the UML class diagram and write a C++ program for this system as described below. At each point marked **[DECIDE]**, you must make an OOP design choice in your code AND write a one-line justification for each as a comment or immediately after the implementation.

(a) Abstract Base Class `Worker`

Define an abstract cl

comment or immediately after the implementation.

(a) Abstract Base Class `Worker`

Define an abstract class `Worker` with protected members:

`workerID`, `workerName`, `basePay`

The class must contain:

* a pure virtual function `showDetails()`
* a virtual function `computePay()`
* an overloaded operator `+` to combine the pay of two workers
* a destructor

[DECIDE 1] Choose:

* whether `operator+` should be a member function or friend function
* and its return type

* a pure virtual function `showDetails()`
* a virtual function `computePay()`
* an overloaded operator `+` to combine the pay of two workers
* a destructor

[DECIDE 1] Choose:

* whether `operator+` should be a member function or friend function
* and its return type

Write a one-line justification.

[DECIDE 2]

Decide whether the destructor should be virtual or non-virtual, and justify your choice.

(b) Derived Classes `FullTimeDeveloper` and `FreelanceDeveloper`

`FullTimeDeveloper`

Add:

* `teamName`

* `performanceBonus`

* `yearsExperience`

Override `computePay()` to include the performance bonus.

`FreelanceDeveloper`

Add:

* `projectCount`

`FreelanceDeveloper`

Add:

* `projectCount`
* `ratePerProject`

Override `computePay()` according to freelance payment rules.

[DECIDE 3]

Choose the inheritance access specifier for both derived classes and justify it.

(c) Class `TechnicalLead`

Derive `TechnicalLead` from both `FullTimeDeveloper` and `FreelanceDeveloper`.

Add:

Choose the inheritance access specifier for both derived classes and justify it.

(c) Class `TechnicalLead`

Derive `TechnicalLead` from both `FullTimeDeveloper` and `FreelanceDeveloper`.

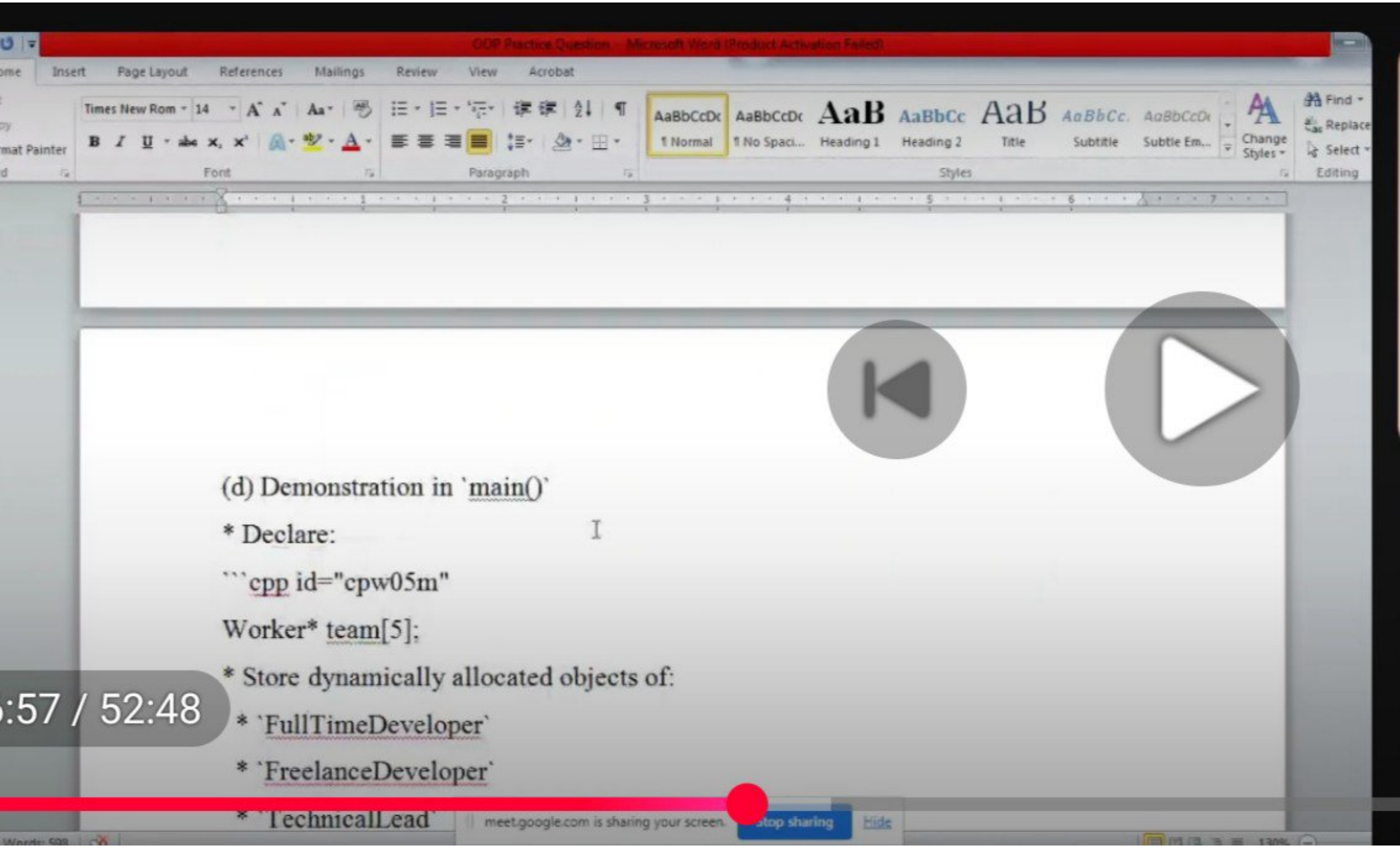
Add:

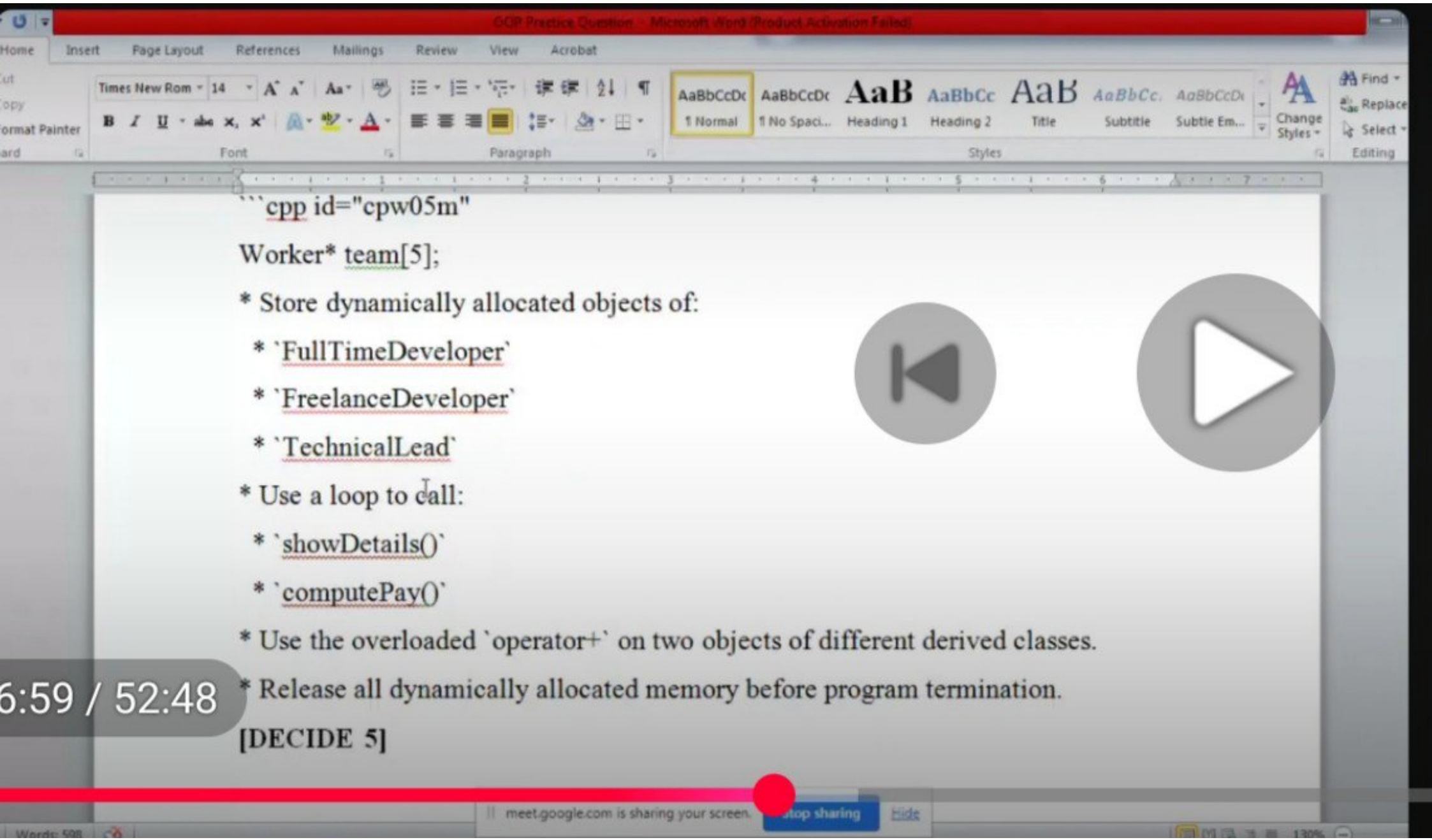
* `leadershipAllowance`

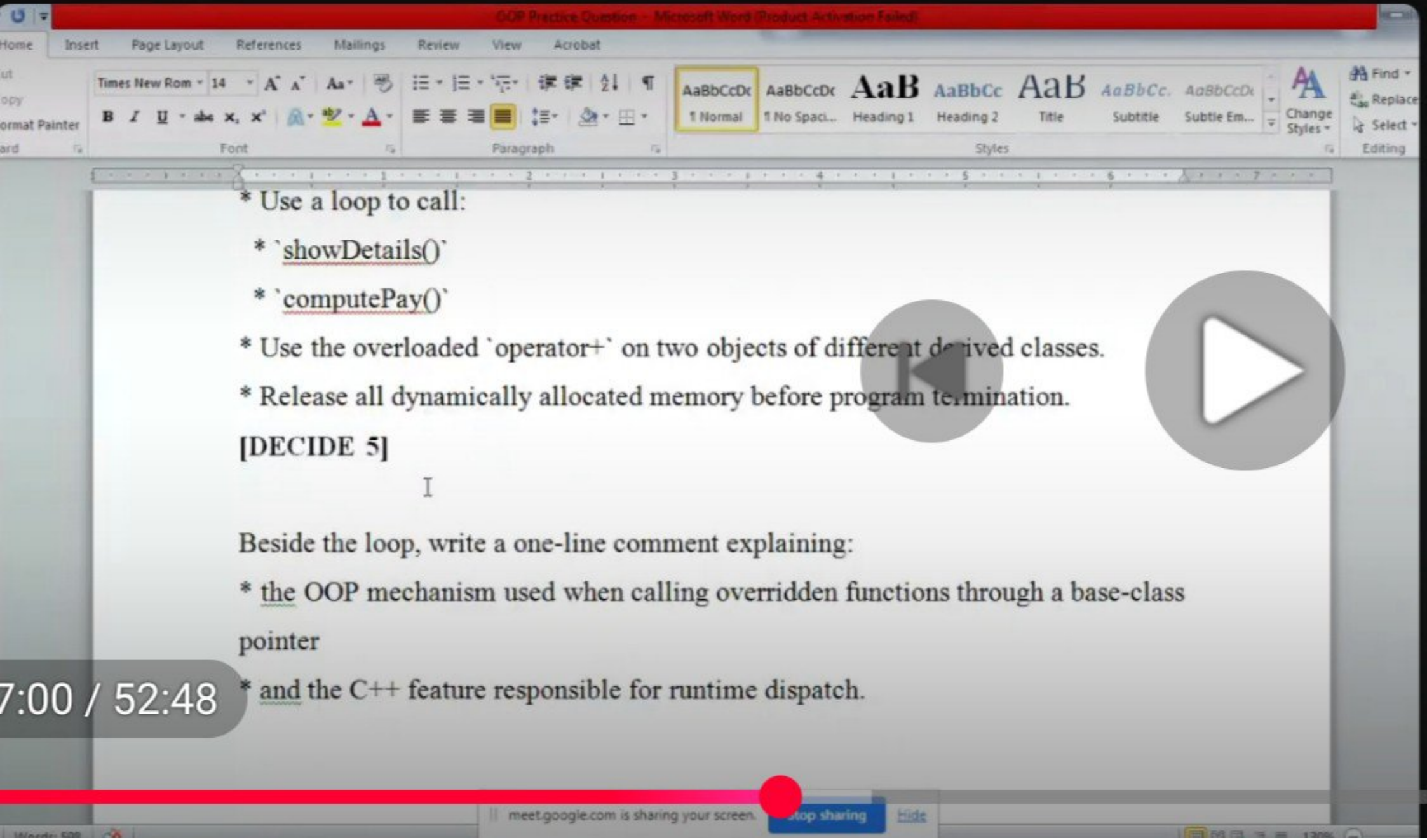
Override `computePay()` and `showDetails()` appropriately.

[DECIDE 4]

Decide whether virtual inheritance should be used when deriving from `Worker`, and justify your answer.


OOP Practice Question - Microsoft Word (Product Activation Failed)
Insert
Page Layout
References
Mailings
Review
View
Acrobat
Times New Rom
14
Font
Paragraph
Styles
Change Styles
Find
Replace
Select
Editing
(d) Demonstration in `main()`
* Declare:
```cpp id="cpw05m"
Worker* team[5];
* Store dynamically allocated objects of:
* `FullTimeDeveloper`
* `FreelanceDeveloper`
* `TechnicalLead`
:57 / 52:48
meet.google.com is sharing your screen.
Stop sharing
Hide
```


GDP Practice Question - Microsoft Word (Product Activation Failed)
Home
Insert
Page Layout
References
Mailings
Review
View
Acrobat
Times New Rom
14
Font
Paragraph
Styles
Change Styles
Find
Replace
Select
Editing
```cpp id="cpw05m"
Worker* team[5];
* Store dynamically allocated objects of:
* `FullTimeDeveloper`
* `FreelanceDeveloper`
* `TechnicalLead`
* Use a loop to call:
* `showDetails()`
* `computePay()`
* Use the overloaded `operator+` on two objects of different derived classes.
* Release all dynamically allocated memory before program termination.
[DECIDE 5]
6:59 / 52:48
meet.google.com is sharing your screen.
Stop sharing
Hide
```



* Use a loop to call:
  * `showDetails()`
  * `computePay()`
* Use the overloaded `operator+` on two objects of different derived classes.
* Release all dynamically allocated memory before program termination.

[DECIDE 5]

Beside the loop, write a one-line comment explaining:

* the OOP mechanism used when calling overridden functions through a base-class pointer
* and the C++ feature responsible for runtime dispatch.